Database Design Document

Boba Bops



Juan Pablo Flores Villarreal
Rowan Banerjee
Christopher Babitzke
Cayden Maguire

Executive Summary

This project proposes a custom, ground-up relational database architecture designed to optimize point-of-sale (POS) and inventory management for a boba cafe. Developed specifically for cashiers and managers, the platform utilizes a graphical user interface to streamline order entry, analyze transaction trends, and automate resupply recommendations.

The core architecture minimizes costs and maximizes efficiency through six interconnected entities, seamlessly linking customer transactions with real-time stock levels as illustrated in the UML diagram.

Employee & Order Management: The Employee entity defines user roles, tracks active clock-ins, and dictates system access levels. This links directly to the Order entity, establishing a chain of responsibility by tracking which cashier handled the transaction, the live status of the order, and the total cost.

Transaction Processing: The OrderItems entity bridges the gap between transactions and the catalog by capturing specific customizations, quantities, and historical pricing at the exact moment of purchase. These items tie directly to the MenuItems entity, which categorizes the shop's offerings, houses descriptions, and dynamically toggles availability.

Automated Inventory Tracking: The Recipes entity serves as a critical junction between the menu and the inventory by detailing the specific raw ingredients required for each drink. When an order is processed, the system automatically calculates the necessary ingredients and decrements the current stock in the Inventory entity. A background task regularly polls this inventory data to flag items falling below their specified reorder threshold.

Successful implementation assumes that the cafe possesses compatible hardware, reliable internet, and functional third-party payment processors. It also assumes staff are adequately trained to input data correctly and that management will proactively maintain accurate product and pricing data. Primary risks include potential database latency disrupting the cashier workflow, as well as human error during order entry or inventory audits. Inaccurate data entry could lead to unexpected resource shortages or inflated order sizes, while shifting stakeholder requirements during development could result in lost progress and delayed timelines.

Purpose

- **Problem Statement/Need** - Cashiers need to be able to enter orders through a graphical user interface, and managers need to be able to use the interface to monitor statistics pertaining to ordering trends. Users should also be able to monitor inventory for every item and create resupply orders at the system’s recommendation.
- **Target Audience** - Our target audience will be both the managers and cashiers for the boba cafe of our choice. The cashiers will need efficient interaction with the inventory database to input orders from customers. Managers will need an organized way to analyze inventories

and recognize trends. Customers do not need to be considered in our target audience because their experience only consists of ordering and receiving a drink, both of which do not depend on the software we develop.

- **Proposed Solution** - Develop a custom, ground-up database system that is optimal for a point-of-sale and inventory management system. By establishing a new relational database architecture, the platform will optimize menu customization, transaction tracking, and inventory control while minimizing costs.

High-Level Entity Design

Employee

- This entity holds each employee as a row and whether or not they are currently active. They also have a role set for them.
- When accessing the POS system, roles determine access and what you can see. A normal employee cannot see the inventory, a manager can.
- This entity will serve to determine the access an employee has.

Order

- This entity holds each order made as rows and how much it cost, and who made the order.
- This is needed otherwise we cannot track when an order is made. Other tables need the actual order, like orderItems, to specify the actual items in the order.
- This will hold orders and be used to specify what order in OrderItems.

OrderItems

- Order items entity is more specific than Order, specifying the actual item bought, quantity, name, and price.
- This is what is in the order and is needed for inventory management. It also holds a foreign key to MenuItems, which more specifically describes the actual Order item.
- This plays an important role as the connection between an Order man, and the specific menu item like Boba.

MenuItems

- This entity will connect Order items to recipes, holding items like Boba or the specific drink. This is also what is categorized, like tea vs coffee.
- This is needed to connect OrderItems to Recipes,

Recipes

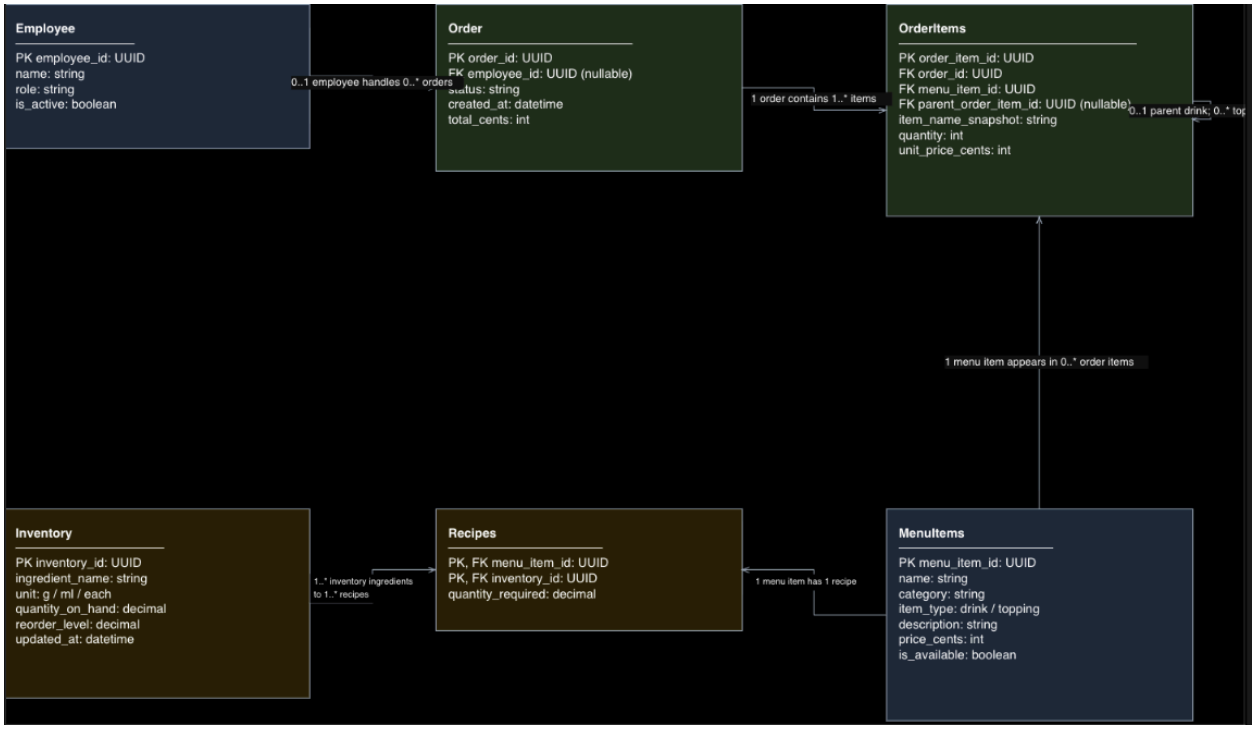
- Connects each menu item to the inventory ingredients required to make it. Each row specifies a single ingredient.
- This entity is needed so that our system knows what ingredients are needed for each menuitem. This is so we don't store it all in menuitem.

- This is the connection between menuitem and inventory. When we order an item from the menu, the system can use its specific recipe to subtract the amount of ingredients from the inventory.

Inventory

- This entity stores each ingredient that we currently have in stock. Boba, syrup, etc. it also stores the quantity we currently have
- Without inventory we wouldn't know when to restock our ingredients. This is needed to track how much we currently have available.
- Inventory updates when orders are complete. The recipe used will determine what ingredients are used, how many, and therefore how much to reduce inventory by. This is crucial for monitoring stock levels.

Low-Level Entity Design



This UML diagram shows six different entities that support the ShareTea boba POS system. We have Employee, Order, OrderItems, MenuItems, Recipes, and Inventory. The employees handle the orders and each order can contain individual items that are linked to the menu. OrderItems also connect toppings to their associated drinks. Each recipe describes what ingredients are needed and the quantity needed for menu items. The inventory tracks what the availability of the ingredients are.

Entity Attributes

Employees will have an ID as they need a unique number to quickly and easily identify them even if there was a case where employees had the same name. They will also have a name for a similar reason: to be able to identify them. They will also have a role attribute so one could understand where in the hierarchy there are in the boba shop. This allows them to have certain permissions and restrictions when interacting with the software. Lastly they will have an is_active attribute to track which employees are currently clocked in and should be in the boba shop working.

Orders will have an ID as they need a unique number to efficiently identify the order. The order will also need the ID number of the employee associated with the order (which will most likely be a cashier) in the event there is an issue with the order that allows one to quickly track the chain of responsibility. The status will allow both the customer and employee to view the somewhat live status of the current order to understand where in the process it is in and if the order can be closed given the status is 'closed' or 'delivered'. It will also have the attribute created_at as a reference for how much time has elapsed since the order has been placed. Lastly, the attribute total_cents is to describe the total cost of the order which will be used for revenue calculations and to display to the customer how much they spent on their order.

OrderItems will have an ID as a unique number to differentiate each line item within an order. It will also need the order_id to group all items belonging to the same customer purchase, as well as the menu_item_id to link back to the actual item being ordered. The parent_order_item_id will be a nullable attribute that links custom additions, such as toppings or bobas, directly to the main drink line item they belong to. The item_name_snapshot captures the exact name of the item at the moment of purchase so historical receipts remain accurate even if menu names change in the future. The quantity attribute tracks how many units of that item or topping were ordered. Lastly, unit_price_cents records the price per item at the time of the transaction.

MenuItems will have an ID as a unique number to distinguish each drink or topping without relying on longer product names. It will have a name attribute so staff and customers can easily identify the item without having to memorize all the unique IDs. The category attribute groups offerings into sections such as milk teas or fruit teas for easy navigation on the POS system. The item_type attribute identifies whether the item is a base drink or a topping. It will also have a description attribute so customers have details of ingredients and flavor profiles. The price_cents attribute defines the exact cost of the item. Lastly, the is_available attribute acts as a toggle to quickly show whether an item is currently in stock and available for purchase.

Recipes will have a menu_item_id and an inventory_id that together form a composite key, linking a specific menu offering directly to every raw ingredient required to make it. This junction structure allows one menu item to require multiple ingredients and one ingredient to be shared across all menu items. Lastly, the quantity_required attribute details the amount of the ingredient needed for a single serving which will automatically deduct stock from inventory whenever an item is ordered or check if the amount needed is higher than what is in stock.

Inventory will have an ID as a unique number to track all ingredients. It will have an ingredient_name to clearly identify what stock item is being tracked. The unit attribute specifies the measurement standard, such as pounds, gallons, or individual counts, to maintain consistency during stock tracking. The quantity_on_hand attribute is a decimal value tracking the current amount of stock remaining in real time. The reorder_level attribute provides a threshold value that alerts employees when stock falls low and requires restocking. Lastly, the updated_at attribute records the precise timestamp of when stock levels were last modified, helping staff maintain auditing accuracy during inventory checks.

Entity Relationships

Employee-Order Relationship: Because an employee can be either idle or handling multiple orders at the same time, there can be 0 or many orders being handled by one employee. Since orders are either queued or processed by up to one employee at a time so as to prevent confusion, there can be 1 employee to an order at a time, or none at all.

Order-OrderItems Relationship: There is no order without the items within, so therefore an order must have at least one contained "OrderItem". There is no maximum size for orders, so there can theoretically be infinitely many OrderItems to one order. Inversely, OrderItems only exist in the context of an order, so there must be at least one order to each OrderItem. OrderItems cannot belong to multiple orders either. For example, while two customers can have separate orders where they both order the boba MenuItem, they can't both order the same specific serving of boba.

OrderItems-MenuItems Relationship: MenuItems refers to different types of items sold at the boba shop, not the item itself. Every boba tea stored and sold within the store belongs to one type of OrderItem, that being the boba tea MenuItem. In this sense, each item sold at the store belongs to one-and-only-one MenuItem type. The MenuItem, on the other hand, can refer to zero physical OrderItems in the store if there are currently none in stock. The MenuItem can also refer to multiple items in the very likely case that there are multiple OrderItems of that type in stock.

MenuItems-Recipe Relationship: Each sold MenuItem has its own specific associated recipe. Each recipe provides instructions for one specific associated menu item. One recipe obviously cannot refer to zero or multiple MenuItems. Each MenuItem needs at least one recipe, and the boba store does not use multiple recipes to make one MenuItem.

Recipe-Inventory Relationship: One inventory item, or ingredient, may be used in multiple different recipes. There is also the possibility that the item could be sold raw without being used in a recipe. For this reason, the inventory of one raw ingredient could be used in anywhere from zero to many recipes. Each recipe has to use at least one ingredient from the inventory, but will likely use many different ingredients from the inventory. One recipe could involve anywhere from one to multiple recipes.

Interactions

When an employee handles an order, the system first verifies that the employee is clocked in by checking Employee.is_active. Then, when verified, a new Order record is instantiated and linked with the employee ID. Line items are added to OrderItems, retrieving names and pricing from MenuItems to create costs. The system computes the sum across all associated OrderItems and records it.

To update stock, the system joins OrderItems with the Recipes table to determine the raw ingredients and the amount for each item. From the OrderItems, it will extract each menu item ID, and then associate them with the correct recipe from the Recipe table to ultimately determine how many ingredients are contained in an order. Then it decrements the quantity in the Inventory table accordingly.

A background task will regularly poll our inventory table to determine which items are out of stock or need to be reordered. Once the correct items are determined, it will set them to not available until the quantity increases above the reorder amount. This completes the process of dynamically relaying the availability of orders to the user.

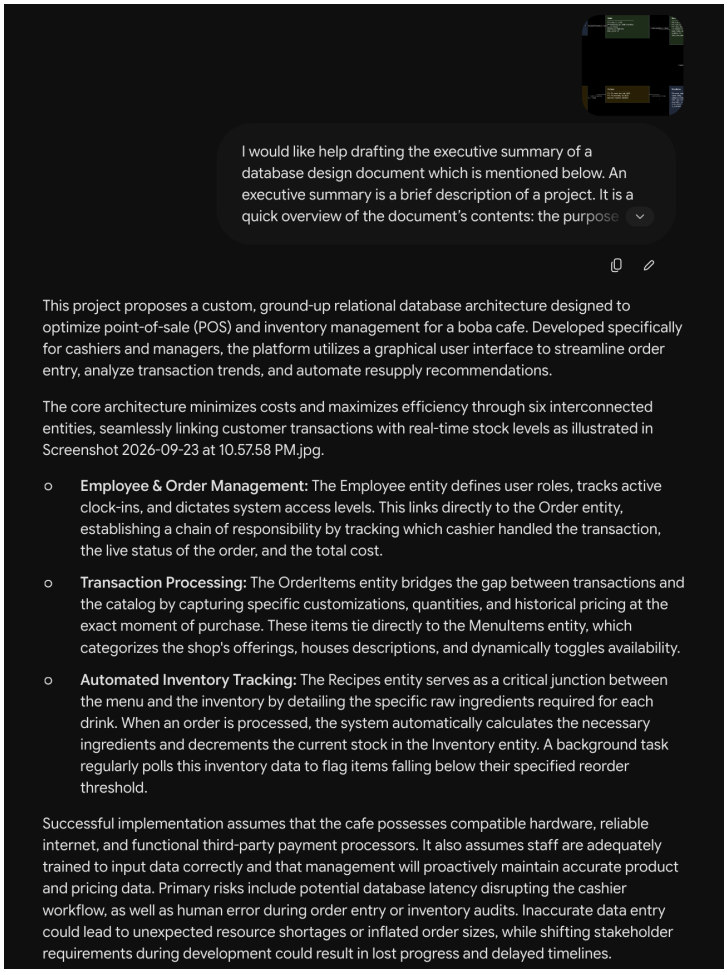
Assumptions and Risks

The proposed solution assumes that all employees have basic computer skills and can be trained to use this system. It also assumes that the employee will enter order information correctly. This system also assumes that managers will always have appropriate permissions to the POS system for updating menu items, prices, inventory, etc. This system also assumes the client will maintain accurate and up to date information on products, ingredients, pricings, toppings, etc. In addition, the solution also assumes the store has compatible hardware, like tablets, computers, receipt printer, cash drawer (if there is cash), etc. The restaurant would also need a reliable internet connection, and third-party payment processors must be working and in usage with the company.

Several risks could arise during the development or implementation of this solution. The ordering workflow could make it more difficult for users to input their orders if the database takes too long to process an instruction or if its design is too complicated. There is additionally the possibility that inventory numbers could be misinput by the user, or exaggerated by some other error. In this case, over-dependence on the system could cause employees of the shop to be blindsided by a lack of resources despite the interface telling them otherwise. Additionally, misinput by the cashier could result in orders accidentally made too large for what the customers, and the shop actually need. Lastly, the design requirements from the stakeholders could change at any time, causing a complete halt or pivot which would result in days or weeks of progress lost.

Appendix: GenAI Tool Usage

- **GenAI Tool Name:** Gemini (https://gemini.google.com/app)
- **Prompts List:**
    - I would like help drafting the executive summary of a database design document which is mentioned below. An executive summary is a brief description of a project. It is a quick overview of the document's contents: the purpose statement, the high-level entity design, the low-level entity design, and the assumptions/risks. The intended audience is a program director (someone who makes decisions about which projects will receive funding), so it should be engaging. Executive summaries are typically 200-400 words. I do not want help with thoughts, ideas, or new information. I will supply all the information. Do not use any other sources besides what I supply in this chat. You can ask me questions if there is information the template needs that I have not yet supplied. If I have not yet supplied it, do not supply it for me. Here is my information: (Pasted the document below)
- **Full Chat History:**



Reflection

Summarize the team's activity during the class workday.

The first 10 minutes of the workday consisted of all members discussing over zoom how to split up the work between us, and laying down some basic problem definitions and identifying the assignment requirements. The remainder of the time was spent by each of us doing our chosen and assigned tasks. In the final ten minutes of the workday, we regrouped to discuss what we would write for this reflection, and discussed any problems we faced in our work.

Identify responsibilities and tasks completed by each team member.

Rowan helped with problem definitions in the first half of the workday, and would later go on to help identify potential risks of using the system.
JP: Organized the group to meet and worked on the problem statement and proposed solution paragraphs.
Chris: Wrote the target audience section and helped with identifying risks.
Cayden: Ideated and drafted the assumptions and risks that could be present in the proposed solution.